

Homework 5 – Review One

1.

```
private Hailstone5() {
}

/**
 * Repeatedly asks the user for a positive integer until the user enters
 * one. Returns the positive integer.
 *
 * @param in
 *         the input stream
 * @param out
 *         the output stream
 * @return a positive integer entered by the user
 */
private static int getPositiveInteger(SimpleReader in, SimpleWriter out) {
    boolean b = false;
    int n = -1;
    do {
        out.println("Enter a positive integer: ");
        String s = in.nextLine();
        if (FormatChecker.canParseInt(s)) {
            n = Integer.parseInt(s);
            if (n > 0) {
                b = true;
            } else {
                out.println("The input should be positive");
            }
        } else {
            out.println("It should be a positive integer");
        }
    } while (!b);
    return n;
}
```

2.

a.

Statement	Variable Values
	i = 2 n = 1
while (i < 5) {	
	i = 2 3 4 n = 1 3 6
n = n + i;	
	i = 2 3 4 n = 3 6 10
i = i + 1;	
	i = 3 4 5 n = 3 6 10
}	
	i = 5 n = 10

b.

Statement	Variable Values
	i = 2 n = 0.5
while (i <= 5) {	
	i = 2 3 4 5 n = 0.5 1 1.33 1.583
n = n + 1.0 / i;	
	i = 2 3 4 5 n = 1 1.33 1.583 1.783
i = i + 1;	
	i = 3 4 5 6 n = 1 1.33 1.583 1.783
}	
	i = 6 n = 1.783

c.

Statement	Variable Values
	x = 1.0 y = 1.0
while (x < 1.8) {	
	x = 1.0 1.5 1.75 y = 1.0 0.5 0.25
y = y / 2.0;	
	x = 1.0 1.5 1.75 y = 0.5 0.25 0.125
x = x + y;	
	x = 1.5 1.75 1.875 y = 0.5 0.25 0.125
}	
	x = 1.875 y = 0.125

d.

Statement	Variable Values
	x = 3 y = 4
while (y > 0) {	
	x = 3 4 5 6 y = 4 3 2 1
x = x + 1;	
	x = 4 5 6 7 y = 4 3 2 1

y = y - 1;	
	x = 4 567 y = 3 210
}	
	x = 7 y = 0

3.

a.

```
int sum = 0;
for (int i = 2; i <= 100; i = i + 2) {
    sum = sum + i;
}
```

b.

```
int sum = 0;
for (int i = 1; i <= 100; i = i + 1) {
    sum = sum + i * i;
}
```

c.

```
int sum = 0;
for (int i = 1; i <= 20; i = i + 1) {
    sum = sum + (int) Math.pow(2, i);
}
```

d.

```
int sum = 0;
for (int i = a; i <= b; i = i + 1) {
    if (i % 2 != 0) {
        sum = sum + i;
    }
}
```

e.

```

int position = 1;
int sum = 0;
while (num > 0) {
    int digit = num % 10;
    if (position % 2 != 0) {
        sum = sum + digit;
    }
    num = num / 10;
    position = position + 1;
}

```

f.

```

String s = String.valueOf(num);
int sum = 0;
for (int i = 0; i < s.length(); i++) {
    if ((i + 1) % 2 != 0) {
        char c = s.charAt(i);
        int digit = Character.getNumericValue(c);
        sum += digit;
    }
}

```

4.

- a. public static int max(int a, int b)
- b. public static double min(double a, double b, double c)
- c. public static boolean isPrime(int n)
- d. public static boolean containString(String a, String b)
- e. public static double balance(double initialBalance, double annualInterest, double years)
- f. public static void printBalance(double initialBalance, double annualInterest, double years)
- g. public static void printCalendar(int month, int year)
- h. public static String weekDay(int day, int month, int year)
- i. public static int getRandom(int n)

5.

When you use a method, the argument values are copied into the formal parameters to initialize them. In addition, when the called method body returns, only the returned value is copied back to the caller. However, the formal parameters won't copy back to the argument values. They will be lost. Therefore, x and y won't be changed after using the method.

6.

a. `public static Boolean allTheSame(int x, int y, int z){
 return x == y && y == z;
}`

b. `public static boolean allDifferent(int x, int y, int z){
 return x != y && y != z && x != z;
}`

c. `public static boolean sorted(int x, int y, int z){
 return x <= y && y <= z;
}`